

Information Retrieval Notes

Academic Year 2026/2027

Contents

1	Introduction and Architecture of Information Retrieval	1
1.1	Definition and Motivations	1
1.1.1	Information Retrieval vs. Data Retrieval	2
1.2	The Two Fundamental Goals: Effectiveness and Efficiency	3
1.2.1	Efficiency and Underlying Hardware Architecture	3
1.3	Information Retrieval System Architecture	4
1.3.1	Text Processing Pipeline	5
1.4	Language Properties and Retrieval Models	5
1.5	The Inverted Index	6
1.5.1	Query Processing and Merge Intersection	6
1.6	Index Compression Techniques	7
1.6.1	Binary Interpolative Coding	8
1.6.2	Dictionary-Based Compression: LZ77	9
1.7	Evolution Toward Neural Information Retrieval	9
1.7.1	Three Embedding Families	10
1.8	k -Approximate Nearest Neighbor Search (k -ANN)	11
2	Evaluation of Information Retrieval Systems	12
2.1	Evaluating Search Engine Quality	12
2.2	Measuring User Happiness	13
2.3	The Cranfield Paradigm and Offline Test Collections	14
2.3.1	Combinatorial Scale Bottlenecks	14
2.3.2	Crowdsourcing: Opportunities and Pitfalls	15
2.3.3	The TREC Infrastructure	15
2.3.4	The Pooling Technique	15
2.4	Information Need vs. Query	16
2.5	Unranked Retrieval Measures	16
2.5.1	Precision, Recall, and the Precision-Recall Trade-off	18
2.5.2	The Harmonic Mean: F-Measure (F_1 -Score)	18
2.5.3	The Weighted F_β -Measure	19
2.6	Rank-Based Measures (Binary Relevance)	19

2.6.1	Precision@k ($P@k$) and Recall@k ($R@k$)	19
2.6.2	Average Precision (AP)	20
2.6.3	Mean Average Precision (MAP)	21
2.6.4	Mean Reciprocal Rank (MRR)	21
2.7	Beyond Binary Relevance: Graded Relevance and NDCG . . .	21
2.7.1	Cumulative Gain (CG) and Discounted Cumulative Gain (DCG)	22
2.7.2	Normalized Discounted Cumulative Gain	22
2.7.3	Complete Numerical Walkthrough from Slides	23
2.7.4	Comparing Alternative Ranking Functions	23

Chapter 1

Introduction and Architecture of Information Retrieval

1.1 Definition and Motivations

Information Retrieval (IR) is the scientific and engineering discipline concerned with representing, storing, organizing, and accessing large collections of predominantly unstructured or semi-structured data to satisfy an **information need** expressed by a user [1, 2].

Definition 1.1.1 (Information Retrieval (Manning et al.)). Information Retrieval is finding material of an unstructured nature (usually text) that satisfies an information need from within large collections stored on computers.

In the modern digital landscape, Information Retrieval extends far beyond general-purpose Web search engines (such as Google or Bing) to serve as the foundational infrastructure for:

- **E-commerce and marketplaces:** product retrieval driven by textual descriptions, structured specifications, and customer reviews.
- **Enterprise and Desktop Search:** indexing and semantic searching across enterprise repositories (documents, emails, source code bases).
- **RAG (*Retrieval-Augmented Generation*) Systems:** grounding Large Language Models (LLMs) with external, authoritative context documents to mitigate hallucinations and ensure up-to-date knowledge.
- **Autonomous agents and recommender engines:** content-based and collaborative filtering for personalized information discovery.

1.1.1 Information Retrieval vs. Data Retrieval

A fundamental conceptual distinction lies in contrasting traditional relational database querying (*Data Retrieval*) with Information Retrieval:

Dimension	Data Retrieval (SQL Databases)	Information Retrieval
Data Structure	Highly structured data with strict relational schemas (tables, types, constraints)	Unstructured or semi-structured data (free text, HTML, PDF, metadata)
Query Language	Formal deterministic queries (e.g., SQL) with exact relational algebra semantics	Natural language keywords or phrases (often ambiguous, short, and incomplete)
Matching Criterion	Exact Boolean matching: a record either strictly satisfies or violates the predicate	Partial and probabilistic matching: documents exhibit varying degrees of relevance
Result Ordering	No intrinsic ordering (unless explicitly specified via ORDER BY)	Central ranking mechanism driven by relevance estimation (<i>Relevance Ranking</i>)
Error Tolerance	Zero tolerance: syntax errors or unexpected values produce errors or empty sets	High tolerance: resilience to typos, synonyms, polysemy, and morphological variants

Table 1.1: Analytical comparison between traditional Data Retrieval and Information Retrieval.

Relevance: The Core of IR

While database systems evaluate binary truth against relational predicates, Information Retrieval focuses on estimating the degree of **relevance** of each document with respect to the user's underlying cognitive information need. Relevance is inherently subjective, context-dependent, and time-varying.

1.2 The Two Fundamental Goals: Effectiveness and Efficiency

A large-scale information retrieval system must simultaneously optimize two competing objectives:

1. **Effectiveness:** Quantifies the intrinsic quality of the returned results, assessing how well the ranked list satisfies the user’s search intent (thoroughly addressed in Chapter 2).
2. **Efficiency:** Quantifies computational cost, throughput (queries processed per second), and latency (demanded within 10–50 milliseconds per query across billions of documents).

1.2.1 Efficiency and Underlying Hardware Architecture

The efficiency of a modern search engine depends not merely on algorithmic complexity, but on hardware execution dynamics:

- **Pipelining and Superscalar Processors:** Modern CPUs segment instruction execution into sequential stages (Instruction Fetch, Decode, Execute, Memory Access, Write Back), issuing multiple instructions per clock cycle.
- **Pipeline Hazards:**
 - *Structural Hazards:* Resource contention when multiple instructions compete for the same execution unit or memory bus.
 - *Data Hazards:* Logical data dependencies between consecutive instructions (RAW: Read-After-Write), forcing pipeline stalls (*bubbles*) when values are unavailable.
 - *Control Hazards:* Branch mispredictions. During posting list intersection loops, an incorrect branch prediction flushes the pipeline, incurring a penalty of 15–20 clock cycles per misprediction.
- **Memory Hierarchy:** CPU registers operate at sub-nanosecond speeds, L1/L2/L3 caches respond in a few clock cycles, whereas main memory (RAM) requires 50–100 ns and secondary storage (NVMe/SSD) demands tens of microseconds. Posting list layouts prioritize sequential, contiguous memory layouts to maximize cache-line reuse and trigger hardware prefetching.

- **SIMD (*Single Instruction Multiple Data*) Vectorization:** Utilizing 128-bit, 256-bit, or 512-bit vector registers (AVX-2, AVX-512, ARM NEON) enables simultaneous comparisons of 8 or 16 document identifiers in a single machine cycle, dramatically accelerating posting intersections.
- **Parallel Computing and Distributed Sharding:** Indices exceeding the capacity of a single machine are partitioned (*sharded*), typically through document-level sharding, allowing compute clusters to process queries concurrently with centralized result aggregation.

1.3 Information Retrieval System Architecture

As depicted in Figure 1.1, an IR system enforces a strict decoupling between the **Offline Pipeline** (indexing and model training) and the **Online Pipeline** (low-latency query serving).

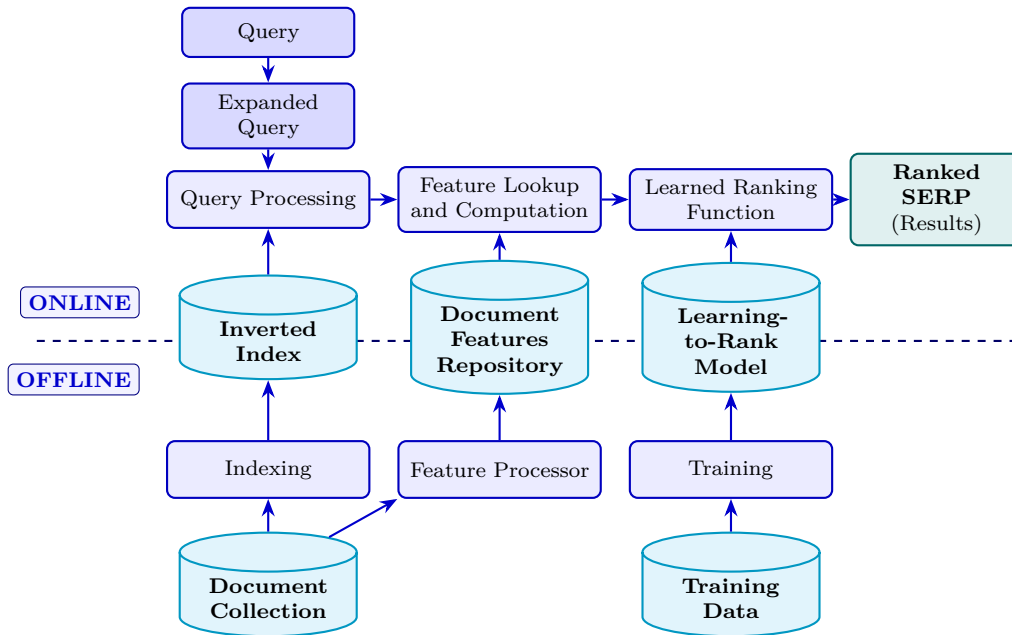


Figure 1.1: Grid architecture of a modern search engine (Online vs. Offline pipelines).

1.3.1 Text Processing Pipeline

Raw textual content undergoes four sequential preparation stages:

1. **Tokenization:** Decomposing character streams into lexical tokens, handling punctuation, special characters, and hyphenated terms.
2. **Normalization and Case Folding:** Converting tokens to lowercase and stripping accents/diacritics to ensure uniformity across queries and documents.
3. **Stopword Filtering:** Removing high-frequency functional words with minimal discriminative value (e.g., "the", "is", "at", "which").
4. **Stemming and Lemmatization:**
 - *Stemming:* Algorithmic stripping of suffixes using heuristic rules (such as Porter's algorithm [1]) to extract the root form (*stem*). Example: "retrieving", "retrieval" → "retriev".
 - *Lemmatization:* Full grammatical analysis utilizing vocabulary dictionaries to map inflected forms to their formal base lemma. Example: "am", "were" → "be".

1.4 Language Properties and Retrieval Models

Natural language word distributions adhere to statistical empirical laws that shape index designs and ranking algorithms:

- **Zipf's Law:** The frequency $f(r)$ of a term is inversely proportional to its frequency rank r :

$$f(r) \propto \frac{1}{r} \quad (1.1)$$

A handful of words dominate overall occurrences, while the vast majority of words appear extremely rarely (*long tail*).

- **Heaps' Law:** The vocabulary size $|V|$ grows as a power law with respect to collection size in terms of total word tokens N :

$$|V| = k \cdot N^\beta \quad (\text{with } 0.4 \leq \beta \leq 0.6) \quad (1.2)$$

The vocabulary never fully saturates as more documents are gathered.

Core retrieval models include:

1. **Boolean Model:** Deterministic matching against logical operators ($\wedge, \vee, \neg$), yielding unranked result sets.
2. **Vector Space Model (TF-IDF):** Documents and queries are modeled as vectors across vocabulary space $|V|$, weighted by term frequency (TF) and inverse document frequency (IDF), scored via cosine similarity.
3. **Probabilistic Model (BM25):** The industry standard for lexical retrieval based on the 2-Poisson indexing model, incorporating non-linear term frequency saturation and document length normalization [2].

1.5 The Inverted Index

The **Inverted Index** reverses document storage directionality, mapping terms to posting lists $t \mapsto \mathcal{P}_t$ [1].

Definition 1.5.1 (Inverted Index). An Inverted Index consists of two complementary structures:

1. **Dictionary (Vocabulary):** Stores all unique terms appearing across the collection, maintained via fast lookup structures (B-Trees or Hash Tables), along with document frequency df_t and a pointer to the start of its posting list.
2. **Posting List:** For each term t , an ordered array of document identifiers sorted strictly in ascending order ($\text{DocID}_1 < \text{DocID}_2 < \dots$). Each posting may also record local term frequencies (tf) and intra-document term position offsets.

1.5.1 Query Processing and Merge Intersection

Maintaining strictly ascending DocID order allows conjunctive (**AND**) queries to be resolved via a synchronized two-pointer merge algorithm (Algorithm 1).

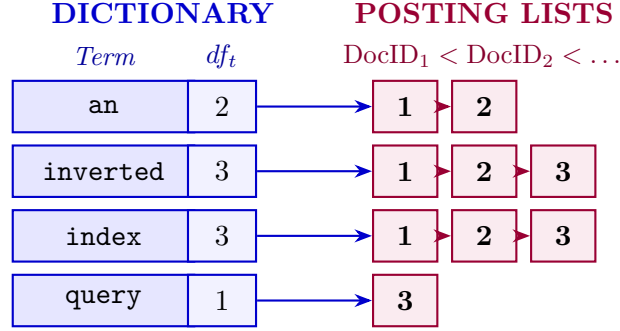


Figure 1.2: Inverted Index structure: Term dictionary with document frequency df_t and linked posting lists ordered by ascending DocID.

Algorithm 1: Merge Intersection for Conjunctive Queries ($P_1 \wedge P_2$)

Input: Two sorted posting lists P_1 and P_2

Output: Sorted list R containing DocIDs present in both lists

```

1  $R \leftarrow \emptyset$ 
2  $i \leftarrow 1; j \leftarrow 1$ 
3 while  $i \leq |P_1|$  and  $j \leq |P_2|$  do
4   if  $P_1[i] == P_2[j]$  then
5      $R.append(P_1[i])$ 
6      $i \leftarrow i + 1$ 
7      $j \leftarrow j + 1$ 
8   else if  $P_1[i] < P_2[j]$  then
9      $i \leftarrow i + 1$ 
10  else
11     $j \leftarrow j + 1$ 
12 return  $R$ 

```

The worst-case runtime complexity is $\mathcal{O}(|P_1| + |P_2|)$. Disjunctive queries (**OR**) compute the union of the posting lists, while Phrase Search checks positional difference constraints across word occurrences.

1.6 Index Compression Techniques

Compressing inverted indices serves two goals: minimizing storage footprint and converting slow I/O bus transfer bottlenecks into fast CPU decompression operations.

1.6.1 Binary Interpolative Coding

Binary Interpolative Coding (introduced by Moffat and Stuiver [3]) is a specialized compression algorithm tailored for strictly monotonically increasing integer sequences.

Definition 1.6.1 (Interpolative Coding Principle). Given a sorted subsequence of n integers $L = [x_1, x_2, \dots, x_n]$ bounded in the interval $[l, h]$ (such that $l \leq x_1 < x_2 < \dots < x_n \leq h$), the algorithm selects the median element $m = \lfloor (n+1)/2 \rfloor$. Because there must be at least $m-1$ distinct integers preceding x_m and at least $n-m$ distinct integers following it, x_m is restricted to the interval:

$$l + m - 1 \leq x_m \leq h - n + m \quad (1.3)$$

The number of valid candidate values for x_m is:

$$R = (h - n + m) - (l + m - 1) + 1 = h - l - n + 1 \quad (1.4)$$

The shifted value $(x_m - (l + m - 1))$ is encoded using exactly $\lceil \log_2 R \rceil$ bits. The procedure then recurses on the left sub-array over $[l, x_m - 1]$ and on the right sub-array over $[x_m + 1, h]$.

Example 1.6.1 (Recursive Decomposition Example from Slides). Consider the sequence of $n = 12$ identifiers within $[0, 62]$:

$$L = [3, 4, 7, 13, 14, 15, 21, 25, 36, 38, 54, 62] \quad (n = 12, m = 6, l = 0, h = 62)$$

The recursive tree partitions the sequence as follows:

- The median splits the array into left sublist $[3, 4, 7, 13, 14]$ with $(n = 5, m = 3, l = 0, h = 14)$ and right sublist $[21, 25, 36, 38, 54]$ with $(n = 5, m = 3, l = 16, h = 62)$.
- At the next depth, $[3, 4, 7, 13, 14]$ splits into $[3, 4]$ with $(n = 2, m = 1, l = 0, h = 6)$ and $[13, 14]$ with $(n = 2, m = 1, l = 8, h = 14)$.
- The leaves encode individual elements: for instance, $[14]$ with $(n = 1, m = 1, l = 14, h = 14)$ yields $R = 14 - 14 - 1 + 1 = 1$, requiring $\lceil \log_2 1 \rceil = 0$ bits!

When postings are densely clustered, R collapses to 1, yielding zero-bit representations for clustered entries.

1.6.2 Dictionary-Based Compression: LZ77

For general text or token streams, the **Lempel-Ziv '77** (LZ77) algorithm maintains a sliding window comprising a Search Buffer and a Lookahead Buffer, emitting triples:

$$(\text{offset}, \text{length}, \text{next_char}) \quad (1.5)$$

For example, given the input sequence:

$$T = a \ b \ a \ b \ a \ a \ a \ a \ b \ a \ b \ a \ a \ a \ a \ b$$

LZ77 generates the pointer sequence:

$$(0, 'a'), (0, 'b'), (2, 3), (1, 3), (7, 8)$$

replacing repeated substrings with compact backward references.

1.7 Evolution Toward Neural Information Retrieval

Beginning in 2018, Transformer architectures and pretrained models like BERT revolutionized Information Retrieval [4, 5]. Neural architectures split into two primary paradigms:

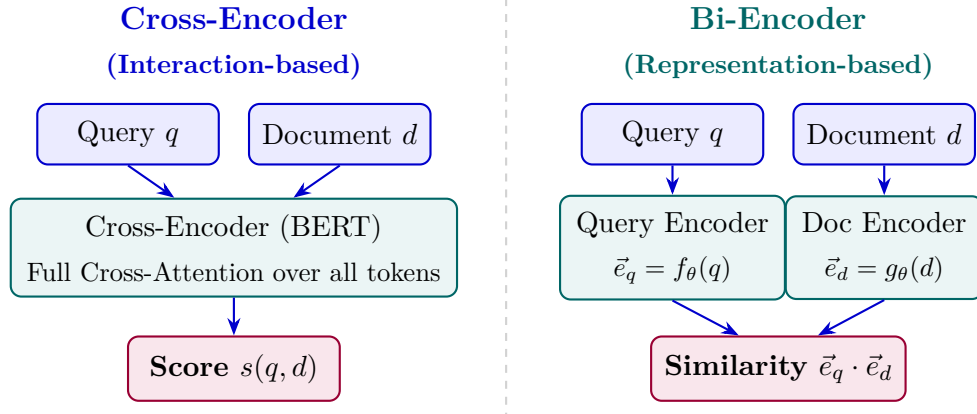


Figure 1.3: Structural comparison between Cross-Encoder (Interaction-based) and Bi-Encoder (Representation-based).

1. **Interaction-Based Models (Cross-Encoders):** The query and document are concatenated into a joint token stream:

$$\text{Input} = [\text{CLS}] \circ q \circ [\text{SEP}] \circ d$$

and fed together through all self-attention layers. This captures fine-grained cross-token interactions, delivering top-tier ranking quality. However, its high inference cost ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$) limits its application to second-stage re-ranking over candidate pools (e.g., top-100 results).

2. **Representation-Based Models (Bi-Encoders):** Query and document are encoded independently into vectors. Document embeddings can be indexed offline, reducing online matching to inner products or vector search.

1.7.1 Three Embedding Families

Representation models fall into three broad categories:

1. **Dense Single-Vector (e.g., DPR, Contriever):** Each document and query is mapped to a single dense vector in $\mathbb{R}^d$ ($d \approx 768$), queried via graph-based vector indices (like HNSW).
2. **Dense Multi-Vector (e.g., ColBERT [4]):** Generates token-level embeddings for query and document tokens, evaluated via **Late Interaction** using the *MaxSim* operator:

$$S(q, d) = \sum_{t_q \in q} \max_{t_d \in d} (\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T) \quad (1.6)$$

Preserves token identity and achieves superior generalization, at the cost of larger index sizes.

3. **Sparse Single-Vector (e.g., SPLADE [5]):** Encodes queries and documents into high-dimensional sparse vectors over the vocabulary $|V|$ ($|V| \approx 30\,000$). Non-zero values represent learned term weights including automated term expansions.
 - **Effectiveness:** High performance across both in-domain and zero-shot out-of-domain evaluation.
 - **Efficiency:** Highly compact, directly indexable inside standard Inverted Index structures.
 - **Interpretability:** Weights align with human-readable vocabulary words, enabling transparent debugging.

1.8 k -Approximate Nearest Neighbor Search (k -ANN)

In high-dimensional embedding spaces, retrieval reduces to finding the top- k documents maximizing the inner product with the query vector (*Maximum Inner Product Search*, MIPS):

$$\operatorname{argmax}_{v \in \mathcal{D}}^{(k)} \vec{q}^T \cdot \vec{v} \quad (1.7)$$

On standard Web-scale collections:

- The document volume exceeds 8.8 million items (e.g., MS MARCO benchmark).
- The vocabulary vector space comprises roughly 30 000 dimensions.
- Each document activates only approximately 150 non-zero components.

High-throughput query serving avoids brute-force linear scans through hybrid forward-inverted sparse vector index architectures, exemplified by modern libraries such as **TusKANNy**, engineered in Rust at the University of Pisa.

Chapter 2

Evaluation of Information Retrieval Systems

2.1 Evaluating Search Engine Quality

Rigorous performance evaluation is essential for designing, fine-tuning, and deploying Information Retrieval systems [1, 2]. Search engine quality is evaluated across two complementary dimensions:

1. Efficiency Metrics:

- *Indexing throughput*: Number of documents processed and incorporated per hour.
- *Incremental indexing*: Ingesting new document streams (e.g., +10 000 products daily) without downtime.
- *Search latency and throughput*: Query processing response time (demanding 95th/99th percentile latencies within 10–50 ms) and CPU/memory utilization under workloads of thousands of queries per second over multi-million or multi-billion document collections.

2. Effectiveness Metrics and Auxiliary Services:

- Quality of related product recommendations, query spell-checking accuracy, and snippet generation readability.
- **Intrinsic ranking effectiveness**: Measures the engine’s core capability to satisfy users by ranking relevant documents at the top of the Search Engine Results Page (SERP).

2.2 Measuring User Happiness

In commercial Web search engines, the overarching objective is maximizing **User Happiness**. Because happiness is an internal cognitive state that cannot be directly queried, production environments rely on observable online behavioral proxies:

- **Clicks and Click-Through Rate (CTR):** The volume and proportion of clicks received by returned results.
 - *Caveat:* Misleading titles and clickbait summaries artificially inflate clicks without truly satisfying the user.
 - *Zero-Click Searches:* When is "no clicks" good news? For factual queries (e.g., weather forecasts, flight schedules, currency conversions, or dictionary definitions), answers rendered directly via *Featured Snippets* or *Knowledge Panels* satisfy the user immediately without requiring any downstream click.
- **Purchases and Economic Conversions:** In e-commerce, the percentage of search sessions resulting in transactions and total monetary spend serve as key indicators.
- **Retention and Repeat Visits:** The fraction of users returning to the search engine within a week or month.
- **Dwell Time:** The duration an attendee spends exploring a clicked page before navigating back to the SERP (*bouncing back*). Extended dwell time indicates genuine engagement, whereas near-instant returns signal dissatisfaction.

Evaluation in the Era of Generative AI and LLMs

The rise of generative conversational engines (such as ChatGPT, SearchGPT, and Perplexity) alters traditional CTR and Dwell Time dynamics: users consume direct synthesized summaries rather than navigating outgoing links. Consequently, factual grounding, hallucination minimization, and citation fidelity supplant traditional link click telemetry.

2.3 The Cranfield Paradigm and Offline Test Collections

While online signals offer valuable telemetry, they are noisy, subject to position bias, and unsuitable for comparing algorithmic improvements offline prior to production deployment. To establish reproducible, controlled evaluation, IR research adheres to the **Cranfield Paradigm**, pioneered by Cyril Cleverdon in the 1960s [6].

Definition 2.3.1 (Test Collection). A standard Information Retrieval test collection is a formal triplet $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ consisting of:

1. A benchmark document collection $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$;
2. A benchmark suite of queries $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$;
3. A set of relevance judgments (*qrels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, assigning a ground-truth relevance assessment (binary or graded) to each query-document pair.

2.3.1 Combinatorial Scale Bottlenecks

Exhaustively assessing all query-document pairs is physically infeasible for realistic corpus sizes. Consider a representative benchmark with:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documents, } |\mathcal{Q}| = 50\,000 \text{ queries}$$

The full Cartesian product comprises:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 50\,000 = 2.5 \times 10^{11} \text{ pairs } (0.25 \text{ trillion judgments})$$

Assuming an experienced human assessor evaluates one document in 2.5 seconds:

$$\begin{aligned} \text{Total Time} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ seconds} \\ &\approx 173\,611\,111 \text{ person-hours} \end{aligned}$$

This would require tens of thousands of human working years! Consequently, benchmark datasets must restrict assessment to a sampled subset of candidate documents.

2.3.2 Crowdsourcing: Opportunities and Pitfalls

One cost-effective alternative delegates judging to online micro-task platforms (e.g., Amazon Mechanical Turk). However, empirical literature indicates that crowdsourced annotations suffer from:

- Substantially higher variance compared to domain experts;
- Inconsistent quality and exposure to adversarial spamming;
- A requirement for robust filtering mechanisms (majority voting, worker reputation scoring, Cohen’s κ inter-rater reliability thresholds).

2.3.3 The TREC Infrastructure

Organized by the National Institute of Standards and Technology (NIST), the **Text REtrieval Conference (TREC)** provides the foundational infrastructure for large-scale IR experimentation. TREC is organized into specialized thematic *tracks* reflecting distinct information access tasks (Web search, biomedical IR, conversational search, deep learning retrieval). The output produced by a participant system executing a retrieval task on a test collection is termed a **Run**.

2.3.4 The Pooling Technique

To select which documents human assessors should judge without evaluating the entire corpus, TREC standardizes the **Pooling** methodology (Figure 2.1):

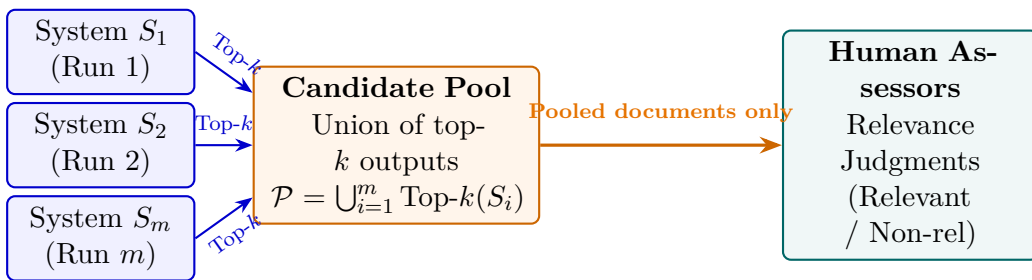


Figure 2.1: The Pooling technique for selecting candidate documents for human judgment.

1. Collect ranked result lists from m diverse retrieval systems for a given topic.

2. Extract the top- k ranked documents from each run (typically $k = 50$ or $k = 100$).
3. Take the **union** of these candidate sets, deduplicating documents to build pool $\mathcal{P}$.
4. Human judges assess relevance strictly for documents inside $\mathcal{P}$.

The Unjudged Document Assumption

All documents in the corpus not present in the pool (*unpooled*) are formally assumed to be **non-relevant** ($r = 0$). While this assumption may theoretically penalize innovative systems surfacing relevant documents overlooked by historical pool contributors, NIST experiments confirm that sufficiently diverse pools maintain stable, fair, and reliable system rankings.

2.4 Information Need vs. Query

A crucial principle in evaluation separates the underlying need from its textual expression:

- **Information Need:** The user's actual cognitive desire. Example: *"My swimming pool bottom is becoming black and needs to be cleaned"*.
- **Query:** The keyword shorthand submitted to the system. Example: "pool cleaner".

Property 2.4.1 (Evaluation Principle). *Document relevance must always be assessed with respect to the user's underlying **Information Need**, never purely against the superficial keywords of the query string.*

2.5 Unranked Retrieval Measures

In unranked evaluation contexts (e.g., Boolean retrieval), the engine returns an unordered set of documents. Performance is derived from the standard contingency table (Table 2.1):

	Document Relevant	Document Non-Relevant
Document Retrieved	True Positive (TP)	False Positive (FP)
Document Not Retrieved	False Negative (FN)	True Negative (TN)

Table 2.1: Contingency matrix for binary query-document classification.

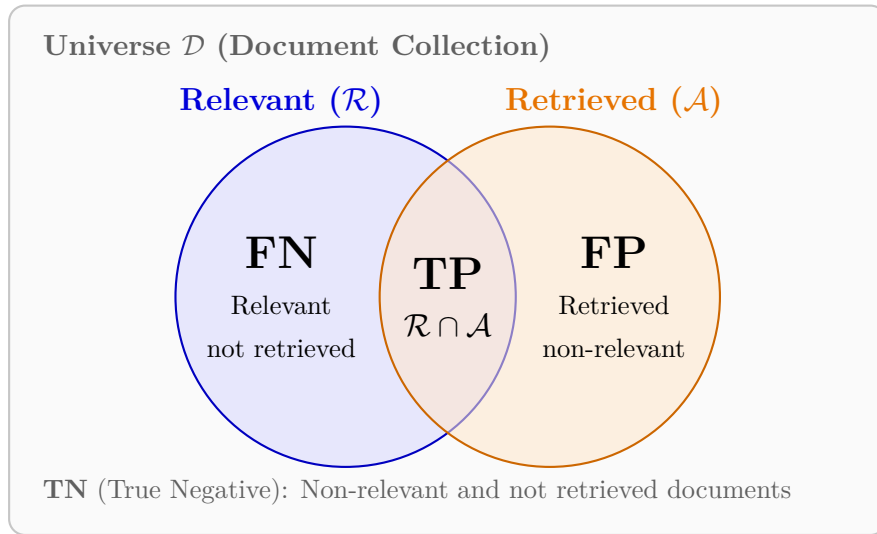


Figure 2.2: Set-theoretic Venn diagram of document retrieval and confusion regions.

2.5.1 Precision, Recall, and the Precision-Recall Trade-off

Definition 2.5.1 (Precision (P)). The fraction of retrieved documents that are relevant:

$$P = \frac{TP}{TP + FP} = P(\text{relevant} \mid \text{retrieved}) \quad (2.1)$$

Definition 2.5.2 (Recall (R)). The fraction of relevant documents in the collection that were successfully retrieved:

$$R = \frac{TP}{TP + FN} = P(\text{retrieved} \mid \text{relevant}) \quad (2.2)$$

An inherent **trade-off** governs Precision and Recall:

- Returning the entire document repository (Retrieved = $\mathcal{D}$) guarantees $R = 100\%$, but drops precision near zero ($P \approx 0$).
- Conversely, returning only the single most confident document can yield $P = 100\%$, at the expense of dismal recall.

2.5.2 The Harmonic Mean: F-Measure (F_1 -Score)

To balance Precision and Recall into a single scalar metric that penalizes extreme disparity, the **F-Measure** calculates their **harmonic mean**:

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (2.3)$$

Property 2.5.1 (Harmonic Mean Properties). *The harmonic mean exhibits key mathematical properties for evaluation:*

1. *It is always upper-bounded by both the geometric and arithmetic means:*

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a + b}{2}$$

2. *When the two operands diverge significantly, the harmonic mean is heavily drawn toward the **minimum** rather than the arithmetic average.*

Example 2.5.1 (Edge Behavior of F-Measure). Consider a degenerate retrieval system returning all documents: $R = 1.0$ and $P = 0.0001$.

- The arithmetic mean yields an artificially inflated score:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- The harmonic mean (F_1) penalizes the system:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

This illustrates why the harmonic mean is mathematically favored over the arithmetic mean.

2.5.3 The Weighted F_β -Measure

When asymmetric priority is given to Precision versus Recall, a weighting parameter $\alpha \in [0, 1]$ is introduced:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad (2.4)$$

Setting $\alpha = \frac{1}{1+\beta^2}$ leads to the standard F_β formulation:

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \quad (2.5)$$

- Values of $\beta < 1$ (e.g., $\beta = 0.5$) emphasize **Precision** over Recall (favored in Web search).
- Values of $\beta > 1$ (e.g., $\beta = 2.0$) emphasize **Recall** over Precision (crucial in legal discovery, patent search, and clinical meta-analyses).

2.6 Rank-Based Measures (Binary Relevance)

Modern search engines return ranked lists ordered by descending relevance scores. Furthermore, Web users rarely look beyond the first page of results ($k = 10$ or $k = 30$).

2.6.1 Precision@k ($P@k$) and Recall@k ($R@k$)

Definition 2.6.1 (Precision@k). Given a fixed rank cutoff $k \in \mathbb{N}^+$, Precision@k is the fraction of relevant documents within the top k positions:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i \quad (2.6)$$

where $\text{rel}_i \in \{0, 1\}$ represents the binary relevance of the document at rank i .

Example 2.6.1 (Computing Precision@k). Consider a ranked list with binary relevance sequence for the top 5 positions $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (note that $P@4 < P@3$)
- $P@5 = \frac{3}{5} = 0.60$

Limitations of $P@k$: It does not reward placing relevant documents higher within the top- k window, and it does not average cleanly across queries with fewer than k total relevant documents. Similarly, *Recall@k* ($R@k$) evaluates the fraction of all relevant documents recovered within the top k .

2.6.2 Average Precision (AP)

Average Precision computes precision at each rank position where a relevant document appears, rewarding systems that place relevant documents earlier in the ranking:

Definition 2.6.2 (Average Precision). For a ranking of N documents corresponding to a query with R total relevant documents in the collection:

$$AP = \frac{1}{R} \sum_{k=1}^N P@k \cdot \text{rel}_k \quad (2.7)$$

Example 2.6.2 (Step-by-Step Calculation from Slides). Consider a query with $R = 3$ total relevant documents. The search engine returns relevant documents at ranks $k_1 = 1$, $k_2 = 3$, and $k_3 = 5$. The corresponding precisions are:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

The resulting Average Precision is:

$$AP = \frac{1}{3} \left(1 + \frac{2}{3} + \frac{3}{5} \right) = \frac{1}{3} (1.00 + 0.667 + 0.600) = \frac{2.267}{3} \approx 0.756 \quad (2.8)$$

2.6.3 Mean Average Precision (MAP)

To quantify overall effectiveness across a suite of $|\mathcal{Q}|$ benchmark queries, we compute the arithmetic mean of their respective AP scores:

Definition 2.6.3 (Mean Average Precision).

$$MAP = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} AP(q) \quad (2.9)$$

Property 2.6.1 (Macro-Averaging in MAP). *MAP implements a **macro-averaging** scheme: each query contributes equally ($1/|\mathcal{Q}|$) regardless of whether it has 2 or 200 relevant documents. It remains the gold standard in academic IR benchmarking for binary relevance.*

2.6.4 Mean Reciprocal Rank (MRR)

In tasks such as known-item search, navigational queries (e.g., locating an institutional homepage), or factoid retrieval, the user is interested solely in finding the **first correct document**. User effort is inversely proportional to the rank of that first hit:

Definition 2.6.4 (Reciprocal Rank and MRR). Let $\text{rank}_1(q)$ denote the rank position of the first relevant document returned for query q . The **Reciprocal Rank** (RR) is:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \quad (2.10)$$

The **Mean Reciprocal Rank** (MRR) across all queries is:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \quad (2.11)$$

If the first relevant hit appears at rank 1, the score is 1.0; if at rank 2, it drops to 0.5; if delayed to rank 10, it falls to 0.1.

2.7 Beyond Binary Relevance: Graded Relevance and NDCG

In practical Web and commercial applications, binary judgments prove inadequate. Documents exhibit varying degrees of utility, typically evaluated on discrete integer scales (e.g., 0 = Non-Relevant, 1 = Marginal, 2 = Relevant, 3 = Highly Relevant).

The **Discounted Cumulative Gain** (DCG) framework, introduced by Järvelin and Kekäläinen [7], rests on two assumptions:

1. Highly relevant documents are much more useful than marginally relevant ones.
2. The lower a document appears in the ranking, the less likely the user is to examine it; hence, gain must be discounted with descending rank.

2.7.1 Cumulative Gain (CG) and Discounted Cumulative Gain (DCG)

Definition 2.7.1 (Cumulative Gain at Rank p). Given a vector of graded relevance values $\mathbf{r} = [r_1, r_2, \dots, r_p]$, the Cumulative Gain is the unweighted sum:

$$CG_p = \sum_{i=1}^p r_i \quad (2.12)$$

CG_p is position-insensitive: swapping a highly relevant document at rank 1 with one at rank 10 leaves CG_p unchanged.

Definition 2.7.2 (Discounted Cumulative Gain (Standard Formulation)). Standard DCG applies a logarithmic discount based on rank i :

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)} \quad (2.13)$$

Commercial search engines (Google, Bing) frequently utilize the **exponential formulation**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (2.14)$$

This formulation places heavy emphasis on surfacing top-tier relevance ($r = 3 \implies 2^3 - 1 = 7$, compared to $r = 1 \implies 2^1 - 1 = 1$).

2.7.2 Normalized Discounted Cumulative Gain

Because different queries have varying numbers of relevant documents, raw DCG_p scores cannot be compared across queries. We normalize by the **Ideal DCG** ($IDCG_p$), obtained by computing the DCG of the ideal ranking sorted by relevance descending:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1] \quad (2.15)$$

2.7.3 Complete Numerical Walkthrough from Slides

Consider the 10-document ranking from the course lecture slides, judged on a scale of 0–3:

$$\mathbf{r} = [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$$

The ideal sorted ranking is:

$$\mathbf{r}^* = [3, 3, 3, 2, 2, 2, 1, 0, 0, 0]$$

Table 2.2 summarizes the step-by-step progression across all 10 ranks.

i	r_i	Discount	Discounted Gain	$DCG@i$	r_i^*	Ideal Gain	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	3.00	3	3.00	3.00	1.00
2	2	$\log_2(2) = 1.000$	2.00	5.00	3	3.00	6.00	0.83
3	3	$\log_2(3) = 1.585$	1.89	6.89	3	1.89	7.89	0.87
4	0	$\log_2(4) = 2.000$	0.00	6.89	2	1.00	8.89	0.76
5	0	$\log_2(5) = 2.322$	0.00	6.89	2	0.86	9.75	0.71
6	1	$\log_2(6) = 2.585$	0.39	7.28	2	0.77	10.52	0.69
7	2	$\log_2(7) = 2.807$	0.71	7.99	1	0.36	10.88	0.73
8	2	$\log_2(8) = 3.000$	0.67	8.66	0	0.00	10.88	0.80
9	3	$\log_2(9) = 3.170$	0.95	9.61	0	0.00	10.88	0.88
10	0	$\log_2(10) = 3.322$	0.00	9.61	0	0.00	10.88	0.88

Table 2.2: Detailed trace of $DCG@i$, $IDCG@i$, and $NDCG@i$ computation for ranks 1 through 10.

2.7.4 Comparing Alternative Ranking Functions

Consider 4 documents $\{d_1, d_2, d_3, d_4\}$ with graded assessments:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

- **Ground Truth (GT):** Ordering $\langle d_4, d_3, d_2, d_1 \rangle$ with relevance vector $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$

$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF₁):** Ordering $\langle d_3, d_4, d_2, d_1 \rangle$ with relevance $[2, 2, 1, 0]$. Because the first two documents share identical relevance ($r = 2$), their permutation incurs no positional penalty:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$

$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordering $\langle d_3, d_2, d_4, d_1 \rangle$ with relevance $[2, 1, 2, 0]$. Here, d_2 ($r = 1$) erroneously outranks d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + 1.2619 = 4.2619$$

$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

This illustrates how NDCG mathematically penalizes rank inversions in direct proportion to the relevance difference.

Bibliography

- [1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.
- [3] Alistair Moffat and Luke Stuiver. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.
- [4] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [5] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.
- [6] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.
- [7] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.